The Omicron Binary Quadratic Form

The Arithmetic Bridge from Delta Law to OMI Addressing, Symbolic Encoding, and 2.5D Projection

Status: Conceptual Whitepaper / v1.0.0-RC1 Alignment Draft
Scope: OMI Binary Quadratic Form, Delta Law, period-8 orbit, prime-73 carrier, Base36 tracker, 240-state bridge, 5-fold hidden root, 4-fold selector face, symbolic character encoding, and browser projection
Core Claim: The Omicron Binary Quadratic Form is the key arithmetic feature that lets OMI convert lawful bitwise state into addressable, symbolic, replayable, and visible geometry.

Abstract

The Omicron Object Model, or OMI, is built around a 128-bit address pointer:

```
omi-S0-S1-S2-S3-S4-S5-S6-S7/prefix
```

The pointer is the carrier. The rule is the validator. The replay path is the memory. The projection is the visible state.

At the center of the architecture is the **Omicron Binary Quadratic Form**:

```
Q(x, y) = 60x² + 16xy + 4y²
```

This form is the arithmetic bridge between:

```
Delta Law bit motion
period-8 orbit
prime-73 decimal carrier
Base36 symbolic tracker
5! hidden packet root
4! visible selector face
240-state bridge
720 semantic sweep
5040 replay ring
DOM/CSSOM 2.5D extrusion
```

The form does not replace the Delta Law. The Delta Law remains the generator of lawful motion:

```
Δ_C(x) = rotl(x,1) ⊕ rotl(x,3) ⊕ rotr(x,2) ⊕ C
```

But the quadratic form is the **projection engine**: it turns the recovered orbit state into a bounded arithmetic surface that can be addressed, folded, displayed, and replayed.

In short:

```
Δ_C produces lawful motion.
Q(x,y) produces lawful shape.
```

---

1. The Only Design Decision: The Delta Law

OMI begins with one bitwise transition law:

```
Δ_C(x) = rotl(x,1) ⊕ rotl(x,3) ⊕ rotr(x,2) ⊕ C
```

This law contains four structural decisions:

```
rotl(x,1)    rotate left by 1
rotl(x,3)    rotate left by 3
rotr(x,2)    rotate right by 2
⊕ C            XOR with a constant
```

And one boundary discipline:

```
mask to width
```

The rotations preserve bits. They are not shifts. No bit falls off the word.

The XOR operation is reversible.

The constant breaks the zero fixed point.

The mask keeps the state bounded inside the chosen word size, canonically 16 bits.

So the core doctrine is:

```
rotations preserve state
XOR preserves reversibility
constant breaks zero collapse
mask preserves boundedness
```

Everything downstream is interpreted as a consequence or projection of this law.

---

2. What the Delta Law Produces

The Delta Law yields a period-8 orbit on the canonical 16-bit state space.

```
period = 8
```

This period points to the decimal carrier:

```
1 / 73 = 0.01369863 01369863 ...
```

The repeating block is:

```
B = [0, 1, 3, 6, 9, 8, 6, 3]
```

The digit sum is:

```
W = 0 + 1 + 3 + 6 + 9 + 8 + 6 + 3 = 36
```

Therefore:

```
W = 36
```

This is the **Base36 orbital tracker width**.

The recovery operation is:

```
divmod(position, 36)
```

This is the first major bridge:

```
period 8 gives the orbit
prime 73 gives the digit block
digit block sum gives W = 36
W = 36 gives Base36 projection
```

---

3. Why the Quadratic Form Is Needed

The Delta Law moves the state.

The system still needs a way to project that state into geometry.

The projection must be:

```
deterministic
bounded
compatible with 4×4 selector surfaces
compatible with 240-state local bridges
compatible with 720 semantic sweeps
usable in DOM/CSSOM translate3d projection
```

The Omicron Binary Quadratic Form provides that projection:

```
Q(x, y) = 60x² + 16xy + 4y²
```

It takes a local pair `(x,y)` and converts it into a weighted arithmetic surface.

In OMI, `x` and `y` may come from:

```
Base36 residue coordinates
Karnaugh torus coordinates
emoji register fields
domino pip pairs
Fano local selectors
word-frame payload slices
```

The quadratic form then computes a depth, bridge index, or projection weight.

---

4. The Coefficients: 60, 16, and 4

The coefficients are not arbitrary inside the OMI model.

They are selected because they are exactly the bridge terms that connect the factorial tower to the byte/nibble projection surface.

```
Q(x, y) = 60x² + 16xy + 4y²
```

4.1 Coefficient 60

```
60 = 5! / 2
60 = 120 / 2
```

So `60` is half of the five-fold packet root.

It also satisfies:

```
60 = 36 + 24
```

That means:

```
60 = Base36 orbit width + 4! selector face
```

So `60` carries both:

```
W = 36
4! = 24
```

This makes `60` the bridge coefficient between Base36 tracking and four-fold projection.

It is also the sexagesimal base, which makes it compatible with OMI's base-60 clock and scheduling language.

In OMI:

```
60 = half-root coefficient
60 = orbit-plus-selector coefficient
60 = clock-compatible scale
```

4.2 Coefficient 16

```
16 = 2⁴
```

This is the full nibble carrier.

A nibble has:

```
4 bits
16 possible states
```

The 240 bridge uses:

```
15 × 16 = 240
```

So `16` is the full local carrier rail.

In OMI:

```
16 = full nibble carrier
```

It is the width that lets the active surface become byte-addressable.

4.3 Coefficient 4

```
4 = 2²
```

This is the four-fold selector base.

It corresponds to:

```
FS / GS / RS / US
4-bit selector logic
```

```
4×4 Karnaugh torus
4-fold visible face
```

In OMI:

```
4 = selector coefficient
```

It is the smallest visible face coefficient.

---

5. The Form as a Bridge Equation

The quadratic form can be factored:

```
Q(x, y) = 60x² + 16xy + 4y²
        = 4(15x² + 4xy + y²)
```

This exposes the four-fold selector factor:

```
4
```

Inside the parentheses:

```
15x² + 4xy + y²
```

we see:

```
15 = active nibble
4  = selector cross-term
1  = identity payload term
```

So the form contains:

```
active nibble
selector cross-term
identity/payload term
```

This makes `Q(x,y)` a controlled way to fold local coordinates into the 240-state bridge.

6. The 4×4 Torus and the 720 Maximum

For local symbolic projection, OMI often uses:

```
x ∈ {0,1,2,3}
y ∈ {0,1,2,3}
```

That is the 4×4 selector surface.

Evaluate the maximum:

```
Q(3,3) = 60·3² + 16·3·3 + 4·3²
       = 60·9 + 16·9 + 4·9
       = 540 + 144 + 36
       = 720
```

Therefore:

```
Q(x,y) ranges from 0 to 720 on the 4×4 local coordinate square.
```

And:

```
720 = 6!
```

So the quadratic form naturally spans the **6! semantic sweep**.

This is a major result:

```
4×4 local selector coordinates
→ Q(x,y)
→ 0..720
→ 6! semantic sweep
```

7. The 5! Extrusion Scale

If we divide by 6:

```
Z = Q(x,y) / 6
```

then at the maximum:

```
Zmax = 720 / 6 = 120
```

And:

```
120 = 5!
```

So:

```
Z = Q(x,y)/6
```

maps the local 4×4 coordinate surface into the five-fold packet-root scale.

In browser projection terms:

```
CSS translate3d Z-depth
= Q(x,y) / 6
```

This is why the form works so well as a 2.5D extrusion engine:

```
Q(x,y) spans 6!
Q(x,y)/6 spans 5!
```

So the geometry has both:

```
visible semantic sweep: 720
hidden packet root: 120
```

---

8. The 240-State Bridge

The 240 bridge is central to OMI:

```
240 = 2×5!
240 = 15×16
240 = 16×16 − 16
240 = 6!/3
```

The quadratic form touches 240 in several ways.

First:

```
60 × 4 = 240
```

So the largest coefficient multiplied by the four-fold selector count equals the bridge:

```
60 × 4 = 240
```

Second:

```
15 × 16 = 240
```

The form's inner active nibble `15` and carrier `16` reproduce the bridge.

Third:

```
720 / 3 = 240
```

The full `Q(3,3)` maximum equals `720`; divided by the three semantic roles of S-P-O, it yields:

```
240 per semantic role
```

Therefore:

```
Q span = 720 = 3 × 240
```

This means the quadratic form spans three 240-state surfaces:

```
subject surface
predicate surface
object surface
```

Canonical reading:

```
Q(x,y) spans the 6! semantic sweep.
Each semantic role receives one 240-state bridge.
```

---

9. The Hidden Five and the Visible Four

OMI distinguishes the hidden packet root from the visible selector face.

The hidden root is:

```
5! = 120
```

The visible selector face is:

```
4! = 24
```

They meet at:

```
240 = 2×5! = 15×16
```

The quadratic form participates in this meeting.

The hidden side:

```
Q(x,y)/6 ∈ 0..120
```

This reaches the `5!` root scale.

The visible side:

```
Q(x,y) mod 240
```

This maps the computed surface into the visible 240-state bridge.

So the same form supports both readings:

```
hidden reading: Q/6 reaches 5!
visible reading: Q mod 240 reaches local240
```

This gives OMI a root/face duality:

```
Q/6 = hidden packet-root scale
Q mod 240 = visible bridge projection
```

---

10. The Base36 Connection

From the Delta Law:

```
W = 36
```

Base36 gives one symbol per orbit tracker position:

```
0..35 → 0..9,A..Z
```

The quadratic form uses Base36 coordinates through local residue extraction:

```
value36 = parseBase36(symbol)
x = value36 mod 4
y = floor(value36 / 4) mod 4
Q(x,y) = 60x² + 16xy + 4y²
```

This turns each Base36 character into a coordinate on the local 4×4 projection surface.

The result can become:

```
depth
color
```

```
route
local240
slot5040
DOM metadata
CSS transform
```

Base36 therefore names the orbit position, while Q gives it shape.

```
Base36 names.
Q projects.
```

---

11. The Important Base36 Bridge Identity

The 240 bridge also decomposes as:

```
240 = 6×36 + 24
```

That is:

```
240 = six full Base36 bands + one 4! selector cap
```

Since:

```
24 = 4!
```

we get:

```
240 = 6×36 + 4!
```

And since:

```
240 = 2×5!
```

we also get:

```
2×5! = 6×36 + 4!
```

This is the Base36 bridge law.

It means:

```
hidden packet root × orientation
=
six visible Base36 orbit bands + four-fold selector face
```

This is one of the most important OMI symbolic encoding equations.

---

12. The 5040 Replay Ring

The replay ring is:

```
5040 = 7!
```

The bridge decomposition gives:

```
5040 = 7 × 720
5040 = 7 × 3 × 240
```

So OMI can address replay slots as:

```
slot5040 = fano7 × 720 + role3 × 240 + local240
```

Where:

```
fano7    ∈ 0..6
role3    ∈ 0..2
local240 ∈ 0..239
```

The quadratic form computes or contributes to `local240`:

```
local240 = Q(x,y) mod 240
```

Thus:

```
slot5040 = fano7 × 720 + role3 × 240 + (Q(x,y) mod 240)
```

This is the operational replay formula.

It connects:

```
Fano point
S-P-O role
quadratic local bridge
5040-slot replay memory
```

---

13. Symbolic Character Encoding

The symbolic character encoding layer exists so that OMI state can be human-readable and machine-scannable.

It includes:

```
Base36
emoji
domino tiles
16-symbol registers
Code16K
JABCode
DOM/CSSOM selectors
```

But the rule is strict:

```
Symbols project the law.
Symbols do not create the law.
```

The quadratic form is the arithmetic engine behind the symbolic projection.

A symbolic character becomes:

```
symbol
→ code point or value
```

```
→ local coordinates
→ Q(x,y)
→ local240
→ slot5040
→ visual projection
```

The authority remains:

```
OMI pointer
Q(S)
Delta orbit
RULES.omi
FACTS.omi
tests
```

---

14. Emoji Register Projection

A 16-emoji register can model the binary16 layout:

```
1 sign symbol
5 exponent symbols
10 explicit payload symbols
+ 1 implicit lead rule
```

OMI interpretation:

```
sign symbol        → fold / polarity
5 exponent symbols → 2^5 = 32 omi--- body
10 payload symbols  → explicit witness
implicit lead       → hidden root / hidden precision
```

The quadratic form enters after the register is decoded:

```
exponentAccumulator → y
significandPrecision → x
Q(x,y) → projection depth
```

This makes the emoji register a visible symbolic memory block.

But it is still a projection.

```
Emoji register shows the state.
It does not authorize the state.
```

---

15. Domino Tile Projection

Domino tiles are naturally pair-coded.

A domino gives:

```
pipA | pipB
```

OMI reads this as:

```
car | cdr
source | target
x | y
subject | object
```

The pipeline is:

```
domino code point
→ orientation
→ pip pair
→ x,y
→ Q(x,y)
→ voxel extrusion
→ DOM/CSSOM projection
```

So a domino tile is a symbolic way to feed the quadratic form.

It is especially useful because `Q(x,y)` is already binary-pair shaped:

```
Q takes two coordinates.
Domino supplies two coordinates.
```

This makes domino tiles a natural symbolic carrier for OMI voxel maps.

16. OMI Binary Quadratic Form vs Quadratic Lexer

There are two related quadratic ideas in OMI.

16.1 The Quadratic Lexer

The lexer validates the 128-bit frame:

```
Q_frame(S) = E_var + E_const
```

It answers:

```
Is this OMI frame valid?
```

16.2 The Binary Quadratic Projection Form

The projection form is:

```
Q_xy(x,y) = 60x² + 16xy + 4y²
```

It answers:

```
Where does this valid state project?
```

They should not be confused.

Canonical distinction:

```
Q_frame(S) validates the instruction envelope.
Q_xy(x,y) projects the decoded state into the 240/720 geometry.
```

Both are quadratic because both use squared error or squared coordinate surfaces to prevent ambiguous cancellation and preserve deterministic structure.

---

17. Branchless Implementation Form

A compact projection implementation:

```
export function omiQuadraticProject(x, y) {
  return (60 * x * x) + (16 * x * y) + (4 * y * y);
}

export function omiLocal240(x, y) {
  return omiQuadraticProject(x, y) % 240;
}

export function omiZRoot(x, y) {
  return omiQuadraticProject(x, y) / 6;
}

export function omiSlot5040(fano7, role3, x, y) {
  const local240 = omiLocal240(x, y);
  return (fano7 * 720) + (role3 * 240) + local240;
}
```

A Base36 projection:

```
const BASE36 = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";

export function projectBase36Symbol(symbol) {
  const v = BASE36.indexOf(symbol.toUpperCase());
  if (v < 0) return null;

  const x = v & 3;
  const y = (v >> 2) & 3;
  const q = omiQuadraticProject(x, y);

  return {
    symbol: symbol.toUpperCase(),
    value36: v,
    x,
    y,
    q,
    local240: q % 240,
    zRoot: q / 6
  };
}
```

For strict runtime, invalid symbols should be rejected or routed through a failure surface. For explanatory projection, returning `null` is sufficient.

---

18. DOM/CSSOM Projection

A browser projection can directly expose the quadratic values:

```
<div
  id="omi-symbol-A"
  data-omi-symbol="A"
  data-omi-q="..."
  data-omi-local240="..."
  data-omi-slot5040="...">
</div>
```

CSSOM can then style OMI state:

```
[data-omi-local240] {
  transform-style: preserve-3d;
}

[data-omi-fold="complement"] {
  filter: invert(1);
}
```

The projection path is:

```
valid pointer
→ decoded symbolic coordinate
→ Q(x,y)
→ local240
→ slot5040
→ DOM/CSSOM
```

This makes the browser a live OMI geometry surface.

---

19. Proposed Canonical Rules

Rule: Binary Quadratic Projection Form

```
# [Rule 0x78]: Omicron Binary Quadratic Projection Form
#   The projection form Q(x,y)=60x²+16xy+4y² MUST be used as the canonical
#   arithmetic bridge from local symbolic coordinates to OMI 240/720 geometry.
omi-0000-0000-0000-0000-0000-0000-0078-0001/128 MUST project-local-state-
through-omi-binary-quadratic-form
```

Rule: Quadratic Frame vs Projection Boundary

```
# [Rule 0x79]: Quadratic Frame/Projection Boundary
#   The frame lexer Q_frame(S)=E_var+E_const MUST validate the 128-bit envelope;
#   the projection form Q_xy(x,y) MUST project decoded local coordinates.
omi-0000-0000-0000-0000-0000-0000-0079-0001/128 MUST distinguish-frame-
validation-from-coordinate-projection
```

Rule: Quadratic Local240 Projection

```
# [Rule 0x7A]: Quadratic Local240 Projection
#   local240 MUST be derivable as Q(x,y) mod 240 for symbolic coordinate
#   projection into the 5040 replay ring.
omi-0000-0000-0000-0000-0000-0000-007a-0001/128 MUST derive-local240-from-
quadratic-projection
```

Rule: Quadratic DOM/CSSOM Projection

```
# [Rule 0x7B]: Quadratic DOM/CSSOM Projection
#   DOM and CSSOM symbolic projection MAY expose Q(x,y), local240, and slot5040
#   metadata, but MUST NOT replace pointer, rule, or test authority.
omi-0000-0000-0000-0000-0000-0000-007b-0001/128 MUST preserve-quadratic-
symbolic-projection-boundary
```

---

20. Proposed Facts

```
# --- Omicron Binary Quadratic Form Facts ---

omi-0000-0000-0000-0000-0000-0000-0078-1001/128 FACT omi-binary-quadratic-form-
documented
omi-0000-0000-0000-0000-0000-0000-0078-1002/128 FACT quadratic-form-spans-six-
factorial-sweep-on-four-by-four-surface
omi-0000-0000-0000-0000-0000-0000-0078-1003/128 FACT quadratic-form-divided-by-
six-spans-five-factorial-root-scale
omi-0000-0000-0000-0000-0000-0000-007a-1001/128 FACT local240-derivable-from-
quadratic-form-modulo-240
omi-0000-0000-0000-0000-0000-0000-007b-1001/128 FACT quadratic-symbolic-dom-
projection-documented
```

---

21. Verification Tests to Add

Suggested tests:

```
test/omi-binary-quadratic-form.test.js
```

Minimum assertions:

```
Q(0,0) = 0
Q(3,3) = 720
Q(x,y) is integer for x,y in 0..3
Q(x,y)/6 ranges within 0..120 for x,y in 0..3
Q(x,y) mod 240 ranges within 0..239
local240 = Q(x,y) mod 240
slot5040 = fano7×720 + role3×240 + local240
slot5040 ranges within 0..5039
Base36 symbol maps to x,y coordinates
symbolic projection does not authorize invalid OMI state
```

These tests make the whitepaper executable.

---

22. Final Cascade

The full OMI cascade is:

```
Delta Law
→ period 8
→ prime 73
→ B = 01369863
→ W = 36
→ Base36 orbit tracker
→ divmod(position,36)
→ local x,y coordinates
→ Q(x,y)=60x²+16xy+4y²
→ local240 = Q mod 240
→ slot5040 = fano7×720 + role3×240 + local240
→ DOM/CSSOM / barcode / emoji / domino projection
```

The hidden-root reading:

```
Q/6 reaches 5! = 120
```

The visible-bridge reading:

```
Q mod 240 reaches local240
```

The semantic sweep reading:

```
Q spans 6! = 720
```

The replay reading:

```
7 × 3 × 240 = 5040
```

Thus the quadratic form is the arithmetic hinge between all major OMI layers.

---

23. Final Position

The OMI Binary Quadratic Form is the key feature because it turns local symbolic coordinates into lawful geometric state.

It connects:

```
bitwise motion
factorial memory
symbolic encoding
visual projection
```

The Delta Law generates orbit.

The quadratic form generates shape.

The 240 bridge bounds projection.

The 5040 ring records replay.

The browser renders visible state.

Therefore:

```
Δ_C is the motion law.
Q(S) is the frame validator.
Q(x,y) is the projection law.
```

Together, they give OMI a complete pipeline:

```
validate
move
project
replay
show
```

---

24. One-Sentence Summary

The Omicron Binary Quadratic Form `Q(x,y)=60x²+16xy+4y²` is the arithmetic projection engine of OMI: it takes local coordinates recovered from the Delta Law's period-8, prime-73, Base36 orbit tracker and maps

them into the 240-state bridge, the 720 semantic sweep, the 120 hidden five-fold root scale, the 5040 replay ring, and finally DOM/CSSOM, emoji, domino, barcode, and browser-visible geometry.